

LAB HANDOUT

Data Preprocessing with Python

Cleaning, Encoding, Scaling and Splitting a Real Dataset for Data Analytics Lab

1. Learning Objectives

By the end of this lab, you will be able to:

- Load a real dataset and inspect its structure, data types and summary statistics.
- Detect and handle missing values using median and mode imputation.
- Convert categorical (text) columns into numbers using label encoding and one-hot encoding.
- Scale numeric features with normalization (Min-Max) and standardization (Z-score).
- Detect and treat outliers using the IQR method.
- Split the cleaned data into training and testing sets, ready for a machine-learning model.

2. Why Data Preprocessing?

Raw data is almost never ready for a machine-learning model. It often contains missing entries, text labels that algorithms cannot read, features on wildly different scales, and extreme values that distort results. Preprocessing is the step that turns messy raw data into clean, numeric, well-scaled data. A common saying captures it well: "garbage in, garbage out." The quality of your model is limited by the quality of the data you feed it.

Setup: Open a fresh Jupyter Notebook or a new Google Colab notebook. All required libraries are pre-installed in Colab and in Anaconda. Run each code block in its own cell, top to bottom, and compare your output with the expected output shown in this handout.

3. Step 1 : Import Libraries & Load the Dataset

Environment check, imports, and loading the Titanic dataset.

We use the Titanic dataset because it contains everything a preprocessing lab needs: missing values, text (categorical) columns, and numeric columns on different scales. It ships inside seaborn, so there is nothing to download.

```
# =====
# STEP 1 - Import libraries & load the dataset
# =====
import numpy as np
import pandas as pd
import seaborn as sns

pd.set_option('display.width', 100)
pd.set_option('display.max_columns', 20)

# Use 8 meaningful raw columns from the Titanic dataset
cols = ['survived', 'pclass', 'sex', 'age',
        'sibsp', 'parch', 'fare', 'embarked']
df = sns.load_dataset('titanic')[cols].copy()

print("Dataset loaded successfully!")
print("Shape (rows, columns):", df.shape)
```

EXPECTED OUTPUT

```
Dataset loaded successfully!
Shape (rows, columns): (891, 8)
```

What the columns mean: survived = target (0 = died, 1 = survived), pclass = ticket class (1/2/3), sex, age, sibsp = #

siblings/spouses aboard, parch = # parents/children aboard, fare = ticket price, embarked = port (C/Q/S).

4. Step 2 : First Look at the Data

View the first rows and understand the layout.

The head() method shows the first five rows. Always eyeball your data first.

```
print(df.head())
```

EXPECTED OUTPUT

	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

5. Step 3: Inspect Structure & Statistics

Use info(), describe(), dtypes and nunique() to profile the data.

3.1 Column types and non-null counts

```
df.info()
```

EXPECTED OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
dtypes: float64(2), int64(4), object(2)
memory usage: 55.8+ KB
```

Read this carefully: age has only 714 non-null values (out of 891) and embarked has 889. Those gaps are the missing values we will fix in Step 4. On very new pandas versions the text columns may show as "str" instead of "object"; both mean the same thing.

3.2 Summary statistics of numeric columns

```
print(df.describe())
```

EXPECTED OUTPUT

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Notice the scale mismatch: age ranges up to 80 while fare ranges up to 512. Models that measure distance (e.g. KNN, SVM) will be dominated by fare unless we scale. That is why Step 6 exists.

3.3 Data types and number of unique values

```
print("Data types:")
print(df.dtypes)
```

```
print("\nUnique values per column:")
print(df.nunique())
```

EXPECTED OUTPUT

```
Data types:
survived      int64
pclass        int64
sex           object
age          float64
sibsp         int64
parch         int64
fare          float64
embarked      object
dtype: object

Unique values per column:
survived      2
pclass        3
sex           2
age           88
sibsp         7
parch         7
fare         248
embarked      3
dtype: int64
```

6. Step 4: Handle Missing Values

Detect missing values, visualize them, then impute.

4.1 Detect missing values

```
print("Missing values per column:")
print(df.isnull().sum())
print("\nMissing percentage (%):")
print((df.isnull().mean() * 100).round(2))
```

EXPECTED OUTPUT

```
Missing values per column:
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
dtype: int64

Missing percentage (%):
survived      0.00
pclass        0.00
sex           0.00
age          19.87
sibsp         0.00
parch         0.00
fare          0.00
embarked      0.22
dtype: float64
```

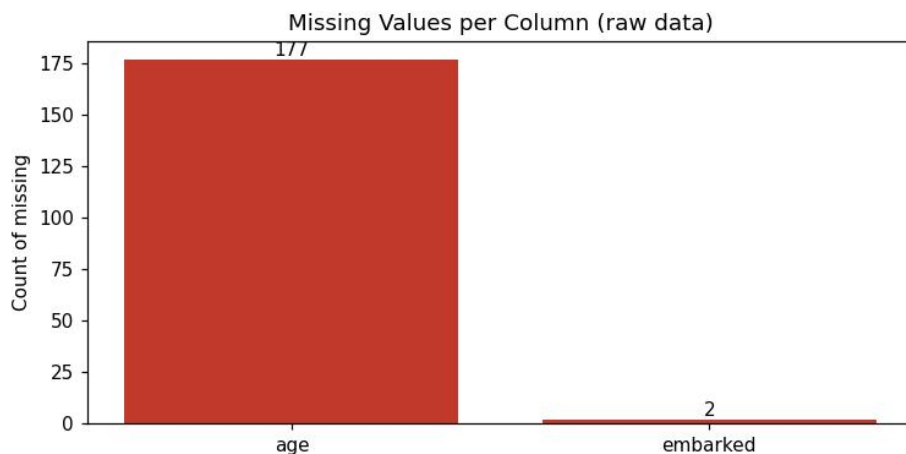
4.2 Visualize the missing values

```
import matplotlib.pyplot as plt

miss = df.isnull().sum()
miss = miss[miss > 0]
plt.figure(figsize=(7, 3.6))
plt.bar(miss.index, miss.values, color='#c0392b')
plt.title('Missing Values per Column (raw data)')
plt.ylabel('Count of missing')
for i, v in enumerate(miss.values):
    plt.text(i, v + 2, str(v), ha='center')
plt.tight_layout()
```

```
plt.show()
```

EXPECTED OUTPUT (A CHART LIKE THIS)



4.3 Impute the missing values

Rule of thumb: fill missing numeric columns with the median (robust to outliers), and missing categorical columns with the mode (most frequent value). We fill age with its median and embarked with its most common port.

```
# Numeric column -> median
df['age'] = df['age'].fillna(df['age'].median())

# Categorical column -> mode (most frequent value)
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

print("After imputation, missing values per column:")
print(df.isnull().sum())
print("\nMedian age used:", df['age'].median())
```

EXPECTED OUTPUT

```
After imputation, missing values per column:
survived    0
pclass      0
sex          0
age          0
sibsp       0
parch       0
fare        0
embarked     0
dtype: int64
```

Median age used: 28.0

Why median, not mean?: The mean is pulled up by a few very old passengers. The median (28) sits in the middle regardless of extremes, so it is the safer choice for skewed columns.

7. Step 5 : Encode Categorical Variables

Turn text columns into numbers with label & one-hot encoding.

Machine-learning algorithms only understand numbers, not text like "male" or "S". We convert categorical columns into numeric form.

5.1 Label encoding (for two categories)

When a column has only two values, we can map them directly to 0 and 1.

```
df['sex_encoded'] = df['sex'].map({'male': 0, 'female': 1})
print(df[['sex', 'sex_encoded']].head())
```

EXPECTED OUTPUT

```

    sex  sex_encoded
0   male           0
1  female           1
2  female           1
3  female           1
4   male           0

```

5.2 One-hot encoding (for three or more categories)

The embarked column has three ports (C, Q, S). We must NOT map them to 0/1/2, because that would falsely tell the model that S is "greater than" C. Instead we create one 0/1 column per category using `pd.get_dummies()`.

```

df = pd.get_dummies(df, columns=['embarked'], prefix='emb', dtype=int)
print(df.head())
print("\nColumns now:", df.columns.tolist())

```

EXPECTED OUTPUT

```

survived  pclass    sex  age  sibsp  parch    fare  sex_encoded  emb_C  emb_Q  emb_S
0         0        3   male  22.0    1      0   7.2500           0      0      0      1
1         1        1  female  38.0    1      0  71.2833           1      1      0      0
2         1        3  female  26.0    0      0   7.9250           1      0      0      1
3         1        1  female  35.0    1      0  53.1000           1      0      0      1
4         0        3   male  35.0    0      0   8.0500           0      0      0      1

```

```

Columns now: ['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
'sex_encoded', 'emb_C', 'emb_Q', 'emb_S']

```

Key idea: One-hot encoding avoids inventing a false order among categories. Each row has exactly one "1" among `emb_C` / `emb_Q` / `emb_S`.

8. Step 6: Feature Scaling

Normalization (Min-Max) and Standardization (Z-score).

Recall the scale mismatch from Step 3.2: age tops out at 80, fare at 512. Scaling brings all numeric features onto a comparable range so no single feature dominates.

6.1 Normalization: Min-Max scaling (range 0 to 1)

```

from sklearn.preprocessing import MinMaxScaler

mm = MinMaxScaler()
df[['age_norm', 'fare_norm']] = mm.fit_transform(df[['age', 'fare']])

print(df[['age', 'age_norm', 'fare', 'fare_norm']].head())
print("\nage_norm range:",
      round(df['age_norm'].min(), 3), "to", round(df['age_norm'].max(), 3))
print("fare_norm range:",
      round(df['fare_norm'].min(), 3), "to", round(df['fare_norm'].max(), 3))

```

EXPECTED OUTPUT

```

age  age_norm  fare  fare_norm
0  22.0  0.271174  7.2500  0.014151
1  38.0  0.472229  71.2833  0.139136
2  26.0  0.321438  7.9250  0.015469
3  35.0  0.434531  53.1000  0.103644
4  35.0  0.434531  8.0500  0.015713

```

```

age_norm range: 0.0 to 1.0
fare_norm range: 0.0 to 1.0

```

Formula: $x_{\text{norm}} = (x - \min) / (\max - \min)$. Every value now sits between 0 and 1.

6.2 Standardization: Z-score scaling (mean 0, std 1)

```

from sklearn.preprocessing import StandardScaler

ss = StandardScaler()
df[['age_std', 'fare_std']] = ss.fit_transform(df[['age', 'fare']])

```

```
print(df[['age', 'age_std', 'fare', 'fare_std']].head())
print("\nage_std    mean=%.4f    std=%.4f" % (df['age_std'].mean(), df['age_std'].std()))
print("fare_std    mean=%.4f    std=%.4f" % (df['fare_std'].mean(), df['fare_std'].std()))
```

EXPECTED OUTPUT

```
   age  age_std  fare  fare_std
0  22.0 -0.565736   7.2500 -0.502445
1  38.0  0.663861  71.2833  0.786845
2  26.0 -0.258337   7.9250 -0.488854
3  35.0  0.433312  53.1000  0.420730
4  35.0  0.433312   8.0500 -0.486337
```

```
age_std    mean=0.0000    std=1.0006
fare_std    mean=0.0000    std=1.0006
```

Which one should I use?: Use Min-Max when you need a fixed 0-1 range (e.g. neural networks, image pixels). Use Standardization when the algorithm assumes roughly Gaussian data (e.g. logistic regression, SVM, PCA). Formula: $z = (x - \text{mean}) / \text{std}$.

9. Step 7: Detect & Handle Outliers

Find extreme fares with the IQR rule, then cap them.

7.1 Detect outliers with the IQR method

The Interquartile Range (IQR) rule flags any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ as an outlier, where $IQR = Q3 - Q1$.

```
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("Q1 =", Q1, " Q3 =", Q3, " IQR =", round(IQR, 4))
print("Lower bound =", round(lower, 4), " Upper bound =", round(upper, 4))

outliers = df[(df['fare'] < lower) | (df['fare'] > upper)]
print("Number of outliers in 'fare':", len(outliers))
```

EXPECTED OUTPUT

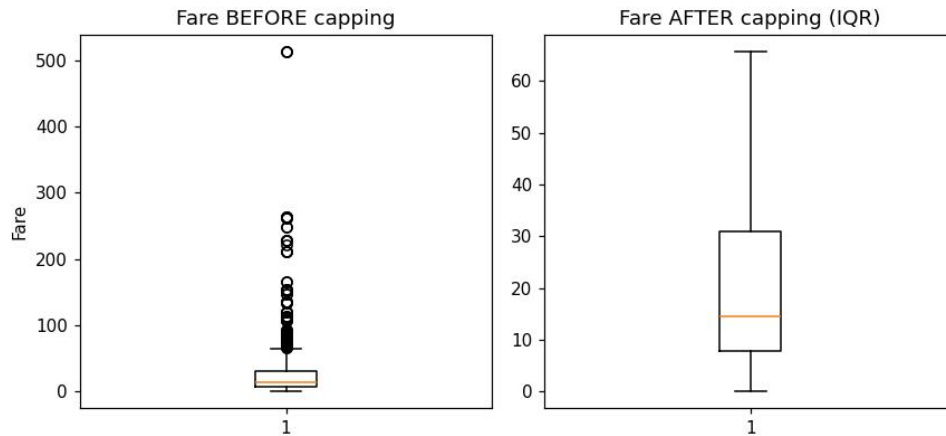
```
Q1 = 7.9104  Q3 = 31.0  IQR = 23.0896
Lower bound = -26.724  Upper bound = 65.6344
Number of outliers in 'fare': 116
```

7.2 Visualize the outliers with a box plot

```
# Capped version for comparison (used in the next step too)
df['fare_capped'] = df['fare'].clip(lower=lower, upper=upper)

fig, ax = plt.subplots(1, 2, figsize=(8, 3.8))
ax[0].boxplot(df['fare']); ax[0].set_title('Fare BEFORE capping')
ax[1].boxplot(df['fare_capped']); ax[1].set_title('Fare AFTER capping (IQR)')
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT (A CHART LIKE THIS)



7.3 Treat outliers by capping (winsorizing)

Rather than delete rows, we cap extreme fares to the boundary value. This keeps every passenger while removing the distorting effect of extreme prices.

```
print("Before capping -> max fare:", df['fare'].max())
print("After capping -> max fare:", df['fare_capped'].max())
print(df[['fare', 'fare_capped']].sort_values('fare', ascending=False).head())
```

EXPECTED OUTPUT

```
Before capping -> max fare: 512.3292
After capping -> max fare: 65.6344
   fare  fare_capped
737  512.3292     65.6344
679  512.3292     65.6344
258  512.3292     65.6344
438  263.0000     65.6344
88   263.0000     65.6344
```

10. Step 8 — Train / Test Split

Separate features (X) and target (y), then split 80/20.

The final preprocessing step: hold out part of the data so we can test the model on data it has never seen. We use an 80% train / 20% test split, and stratify to keep the survived ratio identical in both parts.

```
from sklearn.model_selection import train_test_split

feature_cols = ['pclass', 'sex_encoded', 'age', 'sibsp', 'parch',
                'fare', 'emb_C', 'emb_Q', 'emb_S']
X = df[feature_cols]
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
print("\nTraining target balance:")
print(y_train.value_counts(normalize=True).round(3))
```

EXPECTED OUTPUT

```
X_train shape: (712, 9)
X_test shape: (179, 9)
y_train shape: (712,)
y_test shape: (179,)

Training target balance:
survived
0    0.617
1    0.383
```

Name: proportion, dtype: float64

random_state: Fixing random_state=42 makes the split reproducible you and every classmate get the exact same rows in train and test.

11. Step 9: The Final, Clean Dataset

Let us confirm the data is now fully numeric with zero missing values — ready to feed into any model.

```
final = df[feature_cols + ['survived']]
print("Final preprocessed dataset shape:", final.shape)
print("Any missing values left?", final.isnull().sum().sum())
print(final.head())
```

EXPECTED OUTPUT

```
Final preprocessed dataset shape: (891, 10)
Any missing values left? 0
   pclass  sex_encoded  age  sibsp  parch   fare  emb_C  emb_Q  emb_S  survived
0        3           0  22.0      1      0   7.2500      0      0      1          0
1        1           1  38.0      1      0  71.2833      1      0      0          1
2        3           1  26.0      0      0   7.9250      0      0      1          1
3        1           1  35.0      1      0  53.1000      0      0      1          1
4        3           0  35.0      0      0   8.0500      0      0      1          0
```

12. Quick Reference: What Each Step Did

Step	Problem	Technique used
1-3	Unknown data	Load + info() / describe() / dtypes
4	Missing values	Median (numeric) & mode (categorical) imputation
5	Text columns	Label encoding + one-hot encoding
6	Different scales	Min-Max normalization & Z-score standardization
7	Extreme values	IQR detection + capping
8	Model evaluation	80/20 stratified train-test split

13. Lab Exercises

Work on these using the same df. Write your code and see the output.

1. Instead of the median, fill missing age values with the MEAN. Print the new average age and compare it with the median value (28.0). Which is larger, and why?
2. The pclass column (1, 2, 3) is really a category, not a true number. Apply one-hot encoding to pclass using `pd.get_dummies()` and show the first 5 rows.
3. Apply Min-Max scaling to the sibsp and parch columns. Print the min and max of each scaled column to confirm they lie between 0 and 1.
4. Use the IQR method on the age column (instead of fare). How many age outliers are there? Cap them and print the new maximum age.
5. Change the split to 70% train / 30% test with `random_state=1`. Print the four new shapes.
6. Combine every step above into a single function `preprocess(df)` that returns `X_train`, `X_test`, `y_train`, `y_test`. Test it on a fresh copy of the Titanic dataset.

End of Lab Handout — Data Preprocessing with Python.